UNIT 4: Client Side Scripting [15 hrs]

By: Yuba Raj Devkota

Unit 4: Client-Side Scripting **15 LHs**

Introduction; Adding JavaScript to a Page; Output; Comments; Variables and Data Types; Operators; Control Statements; Functions; Arrays; Classes and Objects; Built-in Objects; Event Handling and Form Validation, Error Handling, Handling Cookies; DOM; BOM; Basics of jQuery, React, and AngularJS, AJAX, and JSON.

What is Scripting Language?

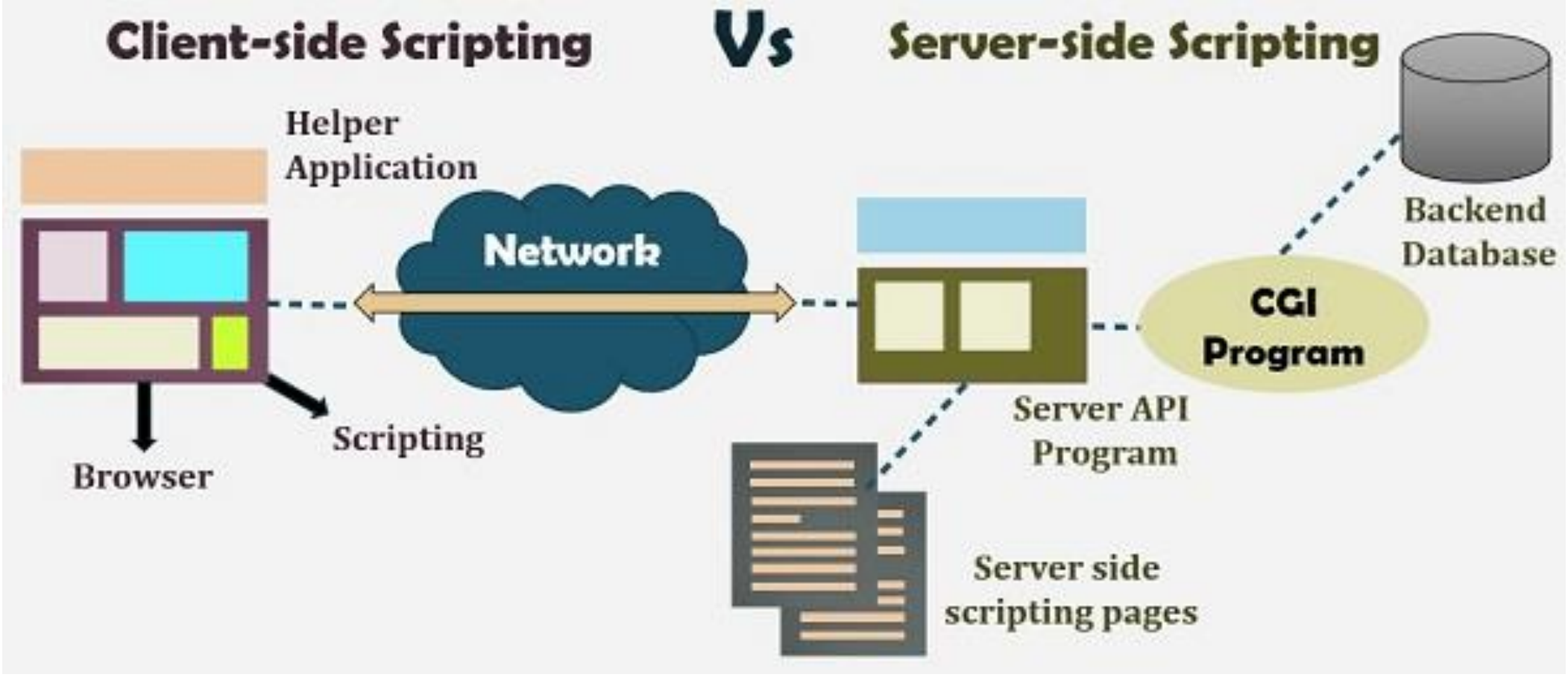
- The scripting language is basically a language where instructions are written for a run time environment. They do not require the compilation step and are rather interpreted.
- A scripting language is a programming language designed for integrating and communicating with other programming languages.
- Some of the Scripting Languages are:
  - **bash**: used in linux platform.
  - **Node js**: Framework to write network applications using Javascript. Some example includes IBM, LinkedIn, Microsoft, Netflix, PayPal, Yahoo for real-time web applications.
  - **Ruby**: Ruby's flexibility has allowed developers to create innovative software. It is a scripting language which is great for web development.
  - **Python**: It is easy, free and open source. It supports procedure-oriented programming and object-oriented programming. Python is an interpreted language with dynamic semantics and huge lines of code are scripted and is currently the most hyped language among developers.
  - **Perl**: A scripting language with innovative features to make it different and popular. Found on all windows and Linux servers. It helps in text manipulation tasks. High traffic websites that use Perl extensively include priceline.com, IMDB.

Scripting Language VS Programming Languages

- Basically, all scripting languages are programming languages.
- The theoretical difference between the two is that scripting languages do not require the compilation step and are rather interpreted. For example, normally, a C program needs to be compiled before running whereas normally, a scripting language like JavaScript or PHP need not be compiled.
- Generally, compiled programs run faster than interpreted programs because they are first converted native machine code.
- Also, compilers read and analyze the code only once, and report the errors collectively that the code might have, but the interpreter will read and analyze the code statements each time it meets them and halts at that very instance if there is some error.
- Some scripting languages traditionally used without an explicit compilation step are JavaScript, PHP, Python, VBScript.
- Some programming languages traditionally used with an explicit compilation step are C, C++.

Client Side Scripting / Javascript

- What is Client Side Scripting vs Server Side Scripting?



Need of Client Scripting Language

The two main benefits of client-side scripting are:

- The user's actions will result in an immediate response because they don't require a trip to the server.
- Fewer resources are used and needed on the web-server.

The two huge advances in client-side scripting are **Ajax** and **jQuery**. Ajax is not a tool or a programing language, but a concept. The concept is to call the server directly from the client without doing a PostBack. The main advantage of Ajax is to save and retrieve data from a database, bypassing the web server, which is known as a **CallBack**.

jQuery is a multi-browser JavaScript library designed to simplify client-side scripting and serves as is a wrapper around JavaScript. jQuery is free open source software.

About Javascript

- Javascript is the most popular scripting language in the world
- It helps you developing great front end as well as back end software using different javascript based frameworks like jQuery, NodeJS etc
- You do not need any special software for javascript because all main web browsers support it.
- You can also create web applications, Mobile Apps and Games using Javascript.
- A very good thing about Javascript is that, in Javascript you can save time by using the already created Libraries and Frameworks

Application of JavaScript

JavaScript (js) is a light-weight object-oriented programming language which is used by several websites for scripting the webpages. There are following features of JavaScript:

- All popular web browsers support JavaScript
- JavaScript follows the syntax and structure of the C programming language. It is a light-weighted and interpreted language.
- It is a case-sensitive language.
- JavaScript is supportable in several operating systems including, Windows, macOS, etc.

Application of JavaScript

- Client-side validation,
- Dynamic drop-down menus,
- Displaying date and time,
- Displaying pop-up windows and dialog boxes
- Displaying clocks etc.

JavaScript Syntax

- Declaring and Assigning Variables
  - var x, y, z; // How to declare variables
    x = 5; y = 6; // How to assign values
    z = x + y; // How to compute values
- Numbers are written with or without decimals:
  - 10.50 or 1001 both are fine
- Strings are text, written within double or single quotes:
  - “Ram Prasad“ or ‘Ram Prasad‘ both are fine
- JavaScript uses arithmetic operators (+ - * /) to compute values:
  - (5 + 6) * 10
- All JavaScript identifiers are **case sensitive**.
  - The variables lastName and lastname, are two different variables:
- Code after double slashes // or between /* and */ is treated as a comment. Comments are ignored, and will not be executed:
  - var x = 5; // It will be executed
    // var x = 6; It will NOT be executed

Lab Exercises

17. Add two numbers entered by user and print the output.
18. Calculate the area of tringle when all sides are known. [s=(a+b+c)/2] and [area = square root (s*(s-a)*(s-b)*(s-c))]
19. Find whether the given number is odd or even
20. write a JavaScript Program to find the largest number among three numbers.
21. Find the Factorial of a Number using JavaScript

Print Hello World

We will use these three ways to print `'Hello, World!'`.

- `console.log()`
- `alert()`
- `document.write()`

1. Using console.log()

`console.log()` is used in debugging the code.

Source Code

```
// the hello world program
console.log('Hello World');
```

2. Using alert()

This method displays an alert box over the current window with the specified message.

```
// the hello world program
alert("Hello, World!");
```

3. Using document.write()

This method used when you want to print the content to the HTML document.

```
// the hello world program
document.write('Hello, World!');
```

Add Two Numbers in JavaScript

Example 1: Add Two Numbers

```
const num1 = 5;
const num2 = 3;

// add two numbers
const sum = num1 + num2;

// display the sum
console.log('The sum of ' + num1 + ' and ' + num2 + ' is: ' + sum);
```

Example 2: Add Two Numbers Entered by the User

```
// store input numbers
const num1 = parseInt(prompt('Enter the first number '));
const num2 = parseInt(prompt('Enter the second number '));

//add two numbers
const sum = num1 + num2;

// display the sum
console.log(`The sum of ${num1} and ${num2} is ${sum}`);
```

Area of tringle when all sides are given

```
// JavaScript program to find the area of a triangle

const side1 = parseInt(prompt('Enter side1: '));
const side2 = parseInt(prompt('Enter side2: '));
const side3 = parseInt(prompt('Enter side3: '));

// calculate the semi-perimeter
const s = (side1 + side2 + side3) / 2;

//calculate the area
const areaValue = Math.sqrt(
  s * (s - side1) * (s - side2) * (s - side3)
);

console.log(
  `The area of the triangle is ${areaValue}`
);
```

Odd or Even number

```
// program to check if the number is even or odd
// take input from the user
const number = prompt("Enter a number: ");

//check if the number is even
if(number % 2 == 0) {
    console.log("The number is even.");
}

// if the number is odd
else {
    console.log("The number is odd.");
}
```

Largest among three numbers

```
// program to find the largest among three numbers

// take input from the user
const num1 = parseFloat(prompt("Enter first number: "));
const num2 = parseFloat(prompt("Enter second number: "));
const num3 = parseFloat(prompt("Enter third number: "));

const largest = Math.max(num1, num2, num3);

// display the result
console.log("The largest number is " + largest);
```

Where to put JavaScript code in HTML?

3 Places to put JavaScript code

- Internal
  - Between the body tag of html
  - Between the head tag of html
- External
  - In .js file (external javaScript)

1) code between the body tag

```
<html>
<body>
<script type="text/javascript">
 alert("Hello Javatpoint");
</script>
</body>
</html>
```

2) JavaScript Example : code between the head tag

```
<html>
<head>
<script type="text/javascript">
function msg(){
 alert("Hello Javatpoint");
}
</script>
</head>
<body>
<p>Welcome to JavaScript</p>
<form>
<input type="button" value="click" onclick="msg()"/>

</form>
</body>
</html>
```

3) External js file

message.js

```
function msg(){
 alert("Hello Javatpoint");
}
```

index.html

```
<html>
<head>
<script type="text/javascript" src="message.js"></script>
</head>
<body>
<p>Welcome to JavaScript</p>
<form>
<input type="button" value="click" onclick="msg()"/>
</form>
</body>
</html>
```

Advantages of External JavaScript

1.It helps in the reusability of code in more than one HTML file.
2.It allows easy code readability.
3.It is time-efficient as web browsers cache the external js files, which further reduces the page loading time.
4.It enables both web designers and coders to work with html and js files parallelly and separately, i.e., without facing any code conflictions.
5.The length of the code reduces as only we need to specify the location of the js file.

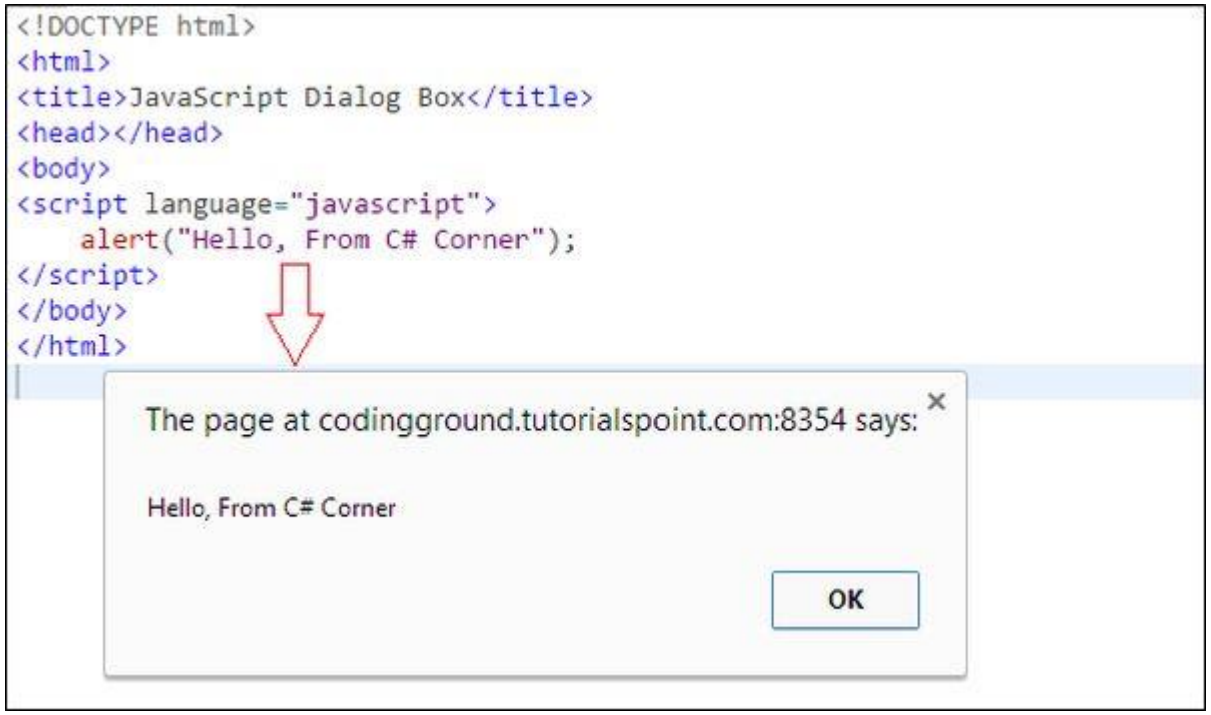
JavaScript Popup Boxes

JavaScript has three kind of popup boxes: Alert box, Confirm box, and Prompt box.

Alert Box: An alert box is often used if you want to make sure information comes through to the user. When an alert box pops up, the user will have to click "OK" to proceed.

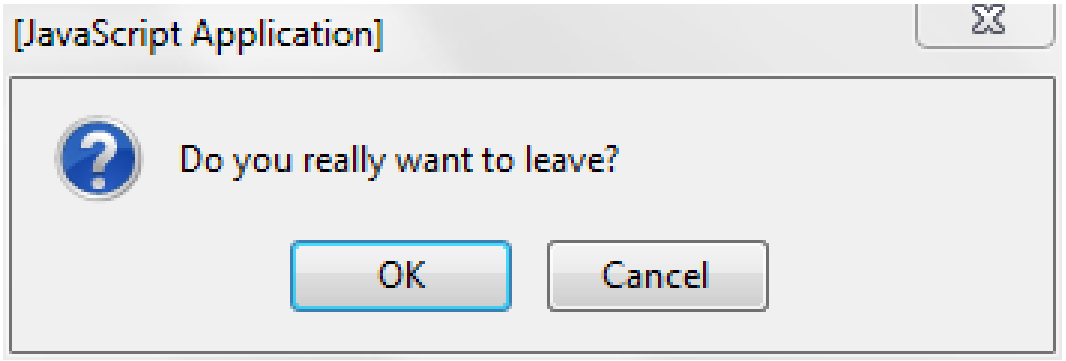
```
alert("I am an alert box!");
```

```
<!DOCTYPE html>
<html>
<title>JavaScript Dialog Box</title>
<head></head>
<body>
<script language="javascript">
    alert("Hello, From C# Corner");
</script>
</body>
</html>
```



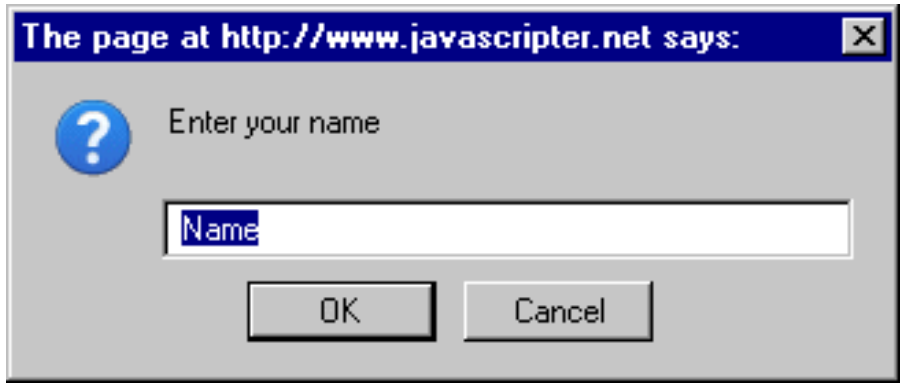
Confirm Box: A confirm box is often used if you want the user to verify or accept something. When a confirm box pops up, the user will have to click either "OK" or "Cancel" to proceed. If the user clicks "OK", the box returns true. If the user clicks "Cancel", the box returns false.

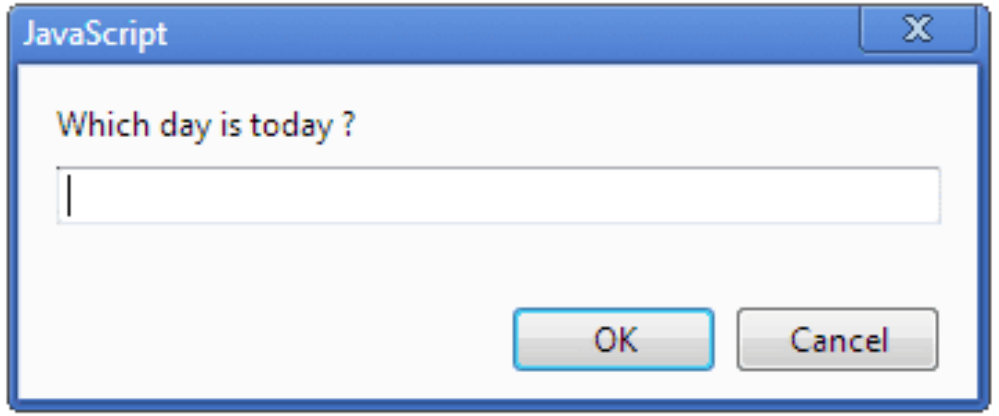
```
if (confirm("Press a button!")) {
  txt = "You pressed OK!";
} else {
  txt = "You pressed Cancel!";
}
```



Prompt Box: A prompt box is often used if you want the user to input a value before entering a page. When a prompt box pops up, the user will have to click either "OK" or "Cancel" to proceed after entering an input value.

```
var person = prompt("Please enter your name", "Harry Potter");

if (person == null || person == "") {
  txt = "User cancelled the prompt.";
} else {
  txt = "Hello " + person + "! How are you today?";
}
```





Javascript Functions

- A JavaScript function is a block of code designed to perform a particular task. A JavaScript function is executed when "something" invokes it (calls it).
- Syntax

```
function myFunction(p1, p2) {
        return p1 * p2;   // The function returns the product of p1 and p2
}
```

Example

Calculate the product of two numbers, and return the result:

```
var x = myFunction(4, 3);   // Function is called, return value will
end up in x

function myFunction(a, b) {
  return a * b;           // Function returns the product of a and b
}
```

The result in x will be: 12

Conditional Statement in JavaScript

In JavaScript we have the following conditional statements:

- Use **if** to specify a block of code to be executed, if a specified condition is true
- Use **else** to specify a block of code to be executed, if the same condition is false
- Use **else if** to specify a new condition to test, if the first condition is false
- Use **switch** to select one of many blocks of code to be executed

The **if** statement specifies a block of code to be executed if a condition is true:

```
if (condition) {
  // block of code to be executed if the condition is true
}
```

The **else** statement specifies a block of code to be executed if the condition is false:

```
if (condition) {
  // block of code to be executed if the condition is true
} else {
  // block of code to be executed if the condition is false
}
```

The **else if** statement specifies a new condition if the first condition is false:

```
if (condition1) {
  // block of code to be executed if condition1 is true
} else if (condition2) {
  // block of code to be executed if the condition1 is false and condition2 is true
} else {
  // block of code to be executed if the condition1 is false and condition2 is false
}
```

If the time is less than 20:00, create a "Good day" greeting, otherwise "Good evening":

```
var time = new Date().getHours();
if (time < 20) {
  greeting = "Good day";
} else {
  greeting = "Good evening";
}
```

If time is less than 10:00, create a "Good morning" greeting, if not, but time is less than 20:00, create a "Good day" greeting, otherwise a "Good evening":

```
var time = new Date().getHours();
if (time < 10) {
  greeting = "Good morning";
} else if (time < 20) {
  greeting = "Good day";
} else {
  greeting = "Good evening";
}
```

Question:

What is the difference between If-else-if ladder and Nested if else?

JavaScript switch Statement

- The switch statement executes a block of code depending on different cases.
- The switch statement is often used together with a break or a default keyword (or both). These are both optional:
  - The **break** keyword breaks out of the switch block. This will stop the execution of more execution of code and/or case testing inside the block. If break is omitted, the next code block in the switch statement is executed.
  - The **default** keyword specifies some code to run if there is no case match. There can only be one default keyword in a switch. Although this is optional, it is recommended that you use it, as it takes care of unexpected cases.

Syntax:

```
switch(expression) {
  case n:
    code block
    break;
  case n:
    code block
    break;
  default:
    default code block
}
```

Example:

```
var text;
switch (new Date().getDay()) {
  case 6:
    text = "Today is Saturday";
    break;
  case 0:
    text = "Today is Sunday";
    break;
  default:
    text = "Looking forward to the Weekend";
}
```

JavaScript Loops

- For Loop
  - var i;
    for (i = 0; i < cars.length; i++) {
      text += cars[i] + "
";
    }
- While Loop
  - while (i < 10) {
      text += "The number is " + i;
      i++;
    }
- Do While Loop
  - do {
      text += "The number is " + i;
      i++;
    }
    while (i < 10);

Create a loop that runs as long as i is less than 10, but increase i with 2 each time.

```
var i = 0;
while (i < 10) {
  console.log(i);
  i = i+2;
}
```

Create a loop that runs through each item in the fruits array.

```
var fruits = ["Apple", "Banana", "Orange"];
for (x of fruits) {
  console.log(x);
}
```

Javascript Array

JavaScript array is an object that represents a collection of similar type of elements. There are 3 ways to construct array in JavaScript

- By array literal
- By creating instance of Array directly (using new keyword)
- By using an Array constructor (using new keyword)

1) JavaScript array literal

Syntax: var arrayname=[value1,value2.....valueN];
values are contained inside [ ] and separated by , (comma).

```
<script>
        var emp=[“Messi",“Ronaldo",“Pele"];

        for (i=0;i<emp.length;i++){
                document.write(emp[i] + "<br/>");
        }
</script>
```

The .length property returns the length of an array.

Output of the above example

Messi
Ronaldo
Pele

2) JavaScript Array directly (new keyword)

var arrayname=new Array();
Here, new keyword is used to create instance of array.

```
<script>
        var i;
        var emp = new Array();
        emp[0] = "Arun";
        emp[1] = "Varun";
        emp[2] = "John";

        for (i=0;i<emp.length;i++){
                document.write(emp[i] + "<br>");
        }
</script>
```

Output

Arun
Varun
John

3) JavaScript array constructor (new keyword)

Here, you need to create instance of array by passing arguments in constructor so that we don't have to provide value explicitly.

```
<script>
        var emp=new Array("Jai","Vijay","Smith");
        for (i=0;i<emp.length;i++){
                document.write(emp[i] + "<br>");
        }
</script>
```

Output

Jai
Vijay
Smith

Array Methods

- Converting Arrays to Strings: The JavaScript method toString() converts an array to a string of (comma separated) array values.
  - var fruits = ["Banana", "Orange", "Apple", "Mango"];
  - document.getElementById("demo").innerHTML = fruits.toString();
  - Result:Banana,Orange,Apple,Mango

- Popping and Pushing: Popping items out of an array, or pushing items into an array.
  - var fruits = ["Banana", "Orange", "Apple", "Mango"];
  - fruits.pop(); // Removes the last element ("Mango") from fruits

  - var fruits = ["Banana", "Orange", "Apple", "Mango"];
  - fruits.push("Kiwi"); // Adds a new element ("Kiwi") to fruits

- Changing Elements: Array elements are accessed using their index number:
  - var fruits = ["Banana", "Orange", "Apple", "Mango"];
  - fruits[0] = "Kiwi"; // Changes the first element of fruits to "Kiwi"

String Methods

- Finding String Length: The length property returns the length of a string:
  - var txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
  - var sln = txt.length;

- Finding a String in a String: The indexOf() method returns the index of (the position of) the first occurrence of a specified text in a string:
  - var str = "Please locate where 'locate' occurs!";
  - var pos = str.indexOf("locate");

- The substring()/substr() Method:
  - var str = "Apple, Banana, Kiwi";
  - var res = str.substring(7, 13);
  - var res1 = str.substr(7, 6);
  - The result of res & res1 will be: Banana

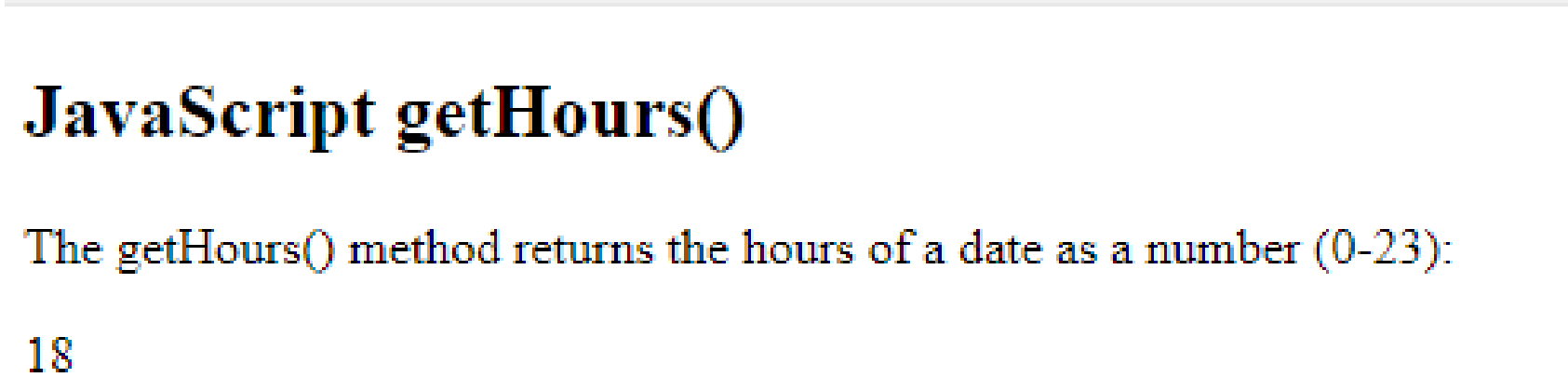
- concat() joins two or more strings:
  - var text1 = "Hello";
  - var text2 = "World";
  - var text3 = text1.concat(" ", text2);

Date Functions in Javascript

- Date objects are created with the new Date() constructor.
- var d = new Date();
- By default, JavaScript will use the browser's time zone and display a date as a full text string:
- **Wed Sep 02 2020 18:23:56 GMT+0545 (Nepal Time)**
- JavaScript Get Date Methods

getFullYear()	Get the **year** as a four digit number (yyyy)
getMonth()	Get the **month** as a number (0-11)
getDate()	Get the **day** as a number (1-31)
getHours()	Get the **hour** (0-23)
getMinutes()	Get the **minute** (0-59)
getSeconds()	Get the **second** (0-59)
getMilliseconds()	Get the **millisecond** (0-999)
getTime()	Get the time (milliseconds since January 1, 1970)
getDay()	Get the weekday as a number (0-6)

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript getHours()</h2>

<p>The getHours() method returns the hours of a date as a number (0-23):</p>

<p id="demo"></p>

<script>
var d = new Date();
document.getElementById("demo").innerHTML = d.getHours();
</script>

</body>
</html>
```



JS HTML DOM (Document Object Model)

- The HTML DOM is a standard **object** model and **programming interface** for HTML. It defines:
  - The HTML elements as **objects**
  - The **properties** of all HTML elements
  - The **methods** to access all HTML elements
  - The **events** for all HTML elements
- In other words: **The HTML DOM is a standard for how to get, change, add, or delete HTML elements.**

Changing HTML Content

The easiest way to modify the content of an HTML element is by using the innerHTML property.

To change the content of an HTML element, use this syntax:

document.getElementById(id).innerHTML = new HTML

```
<html>
<body>

<p id="p1">Hello World!</p>

<script>
document.getElementById("p1").innerHTML = "New text!";
</script>

</body>
</html>
```

Changing the Value of an Attribute

To change the value of an HTML attribute, use this syntax:
document.getElementById(*id*).*attribute = new value*

Example

```
<!DOCTYPE html>
<html>
<body>

<img id="myImage" src="smiley.gif">

<script>
document.getElementById("myImage").src = "landscape.jpg";
</script>

</body>
</html>
```

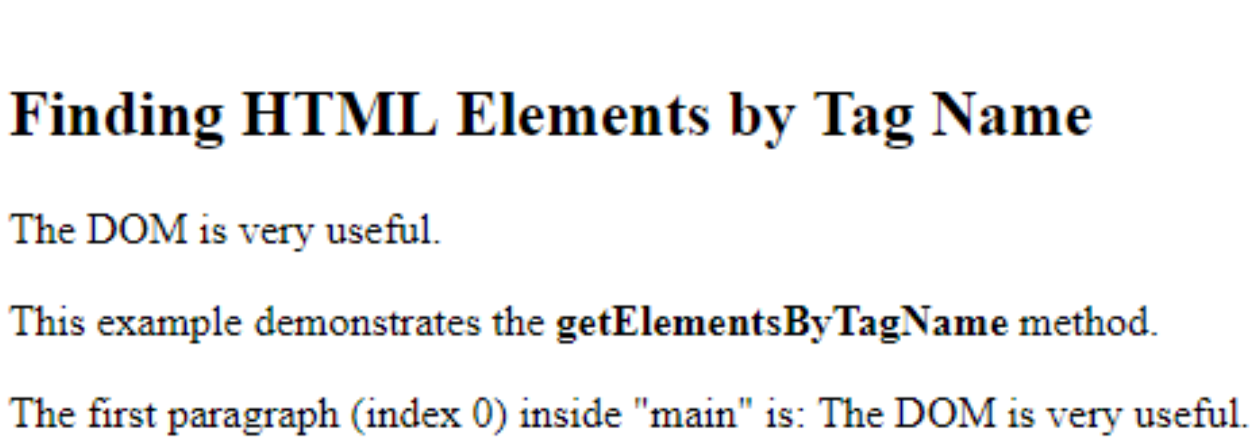
Finding HTML Elements using DOM

- Often, with JavaScript, you want to manipulate HTML elements. To do so, you have to find the elements first. There are several ways to do this:
  - Finding HTML elements by id
  - Finding HTML elements by tag name
  - Finding HTML elements by class name
  - Finding HTML elements by CSS selectors
  - Finding HTML elements by HTML object collections

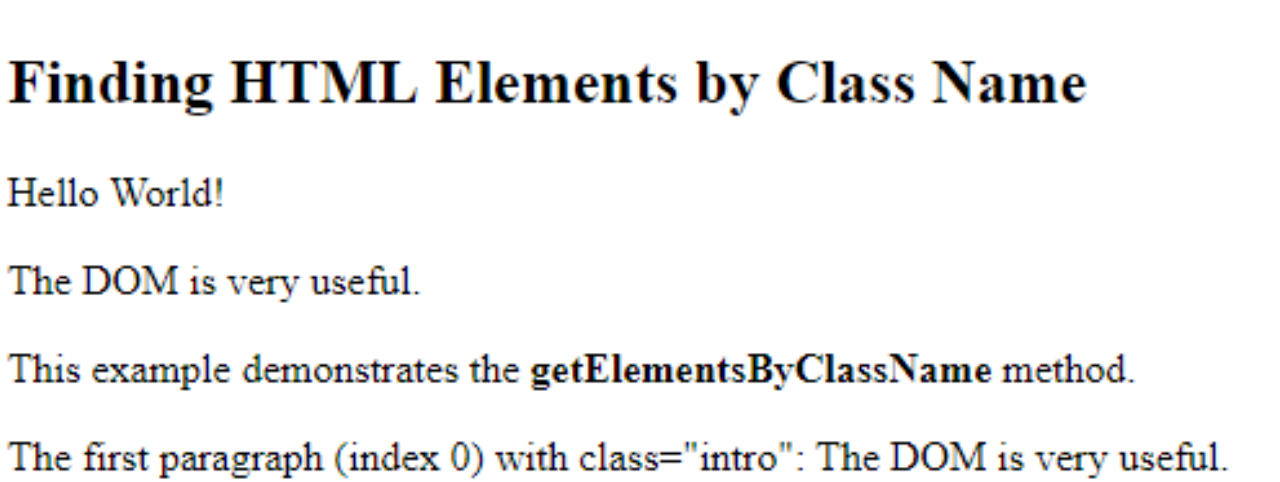
The easiest way to find an HTML element in the DOM, is by using the element id, Tag Name or Class Name

```
var x = document.getElementById("main");
var y = x.getElementsByTagName("p");
var x = document.getElementsByClassName("intro");
```

```
<!DOCTYPE html>
<html>
<body>

<h2>Finding HTML Elements by Tag Name</h2>

<div id="main">
<p>The DOM is very useful.</p>
<p>This example demonstrates the <b>getElementsByTagName</b> method.</p>
</div>

<p id="demo"></p>

<script>
var x = document.getElementById("main");
var y = x.getElementsByTagName("p");
document.getElementById("demo").innerHTML =
'The first paragraph (index 0) inside "main" is: ' + y[0].innerHTML;
</script>

</body>
</html>
```



```
<!DOCTYPE html>
<html>
<body>

<h2>Finding HTML Elements by Class Name</h2>

<p>Hello World!</p>

<p class="intro">The DOM is very useful.</p>
<p class="intro">This example demonstrates the <b>getElementsByClassName</b>
method.</p>

<p id="demo"></p>

<script>
var x = document.getElementsByClassName("intro");
document.getElementById("demo").innerHTML =
'The first paragraph (index 0) with class="intro": ' + x[0].innerHTML;
</script>

</body>
</html>
```



- What are Cookies?
  - Cookies are data, stored in small text files, on your computer. When a web server has sent a web page to a browser, the connection is shut down, and the server forgets everything about the user. Cookies were invented to solve the problem "how to remember information about the user":
  - When a user visits a web page, his/her name can be stored in a cookie. Next time the user visits the page, the cookie "remembers" his/her name.
- Cookies are saved in name-value pairs like:
  - username = John Doe
- Create a Cookie with JavaScript
  - JavaScript can create, read, and delete cookies with the document.cookie property.
  - document.cookie = "username=John Doe";
- Read a Cookie with JavaScript
  - With JavaScript, cookies can be read like this:
  - var x = document.cookie;
- Delete a Cookie with JavaScript
  - You don't have to specify a cookie value when you delete a cookie.
  - Just set the expires parameter to a passed date:
  - document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;";
  - You should define the cookie path to ensure that you delete the right cookie. document.cookie will return all cookies in one string much like: cookie1=value; cookie2=value; cookie3=value;

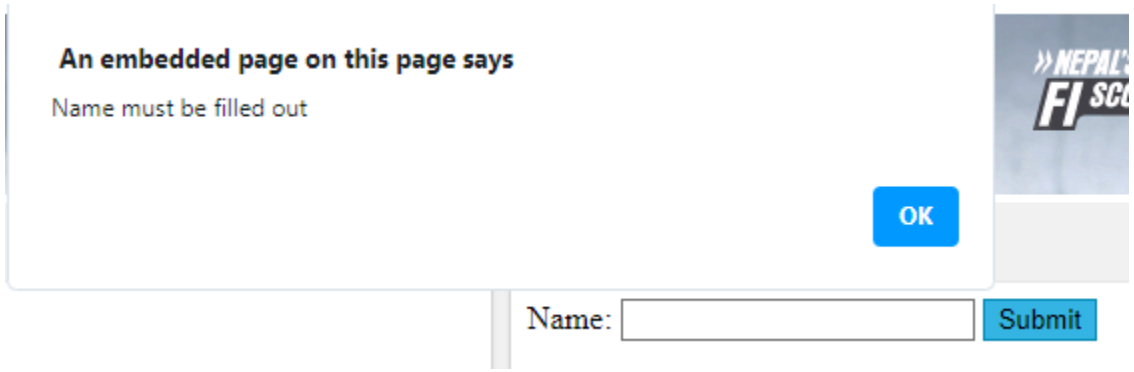
```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
<input type="button" value="setCookie" onclick="setCoo
kie()">
<input type="button" value="getCookie" onclick="getCo
okie()">
   <script>
   function setCookie()
   {
      document.cookie="username=Duke Martin";
   }
   function getCookie()
   {
      if(document.cookie.length!=0)
      {
      alert(document.cookie);
      }
      else
      {
      alert("Cookie not available");
      }
   }
   </script>

</body>
</html>
```

Cookie Example

JavaScript Form Validation

JavaScript Example

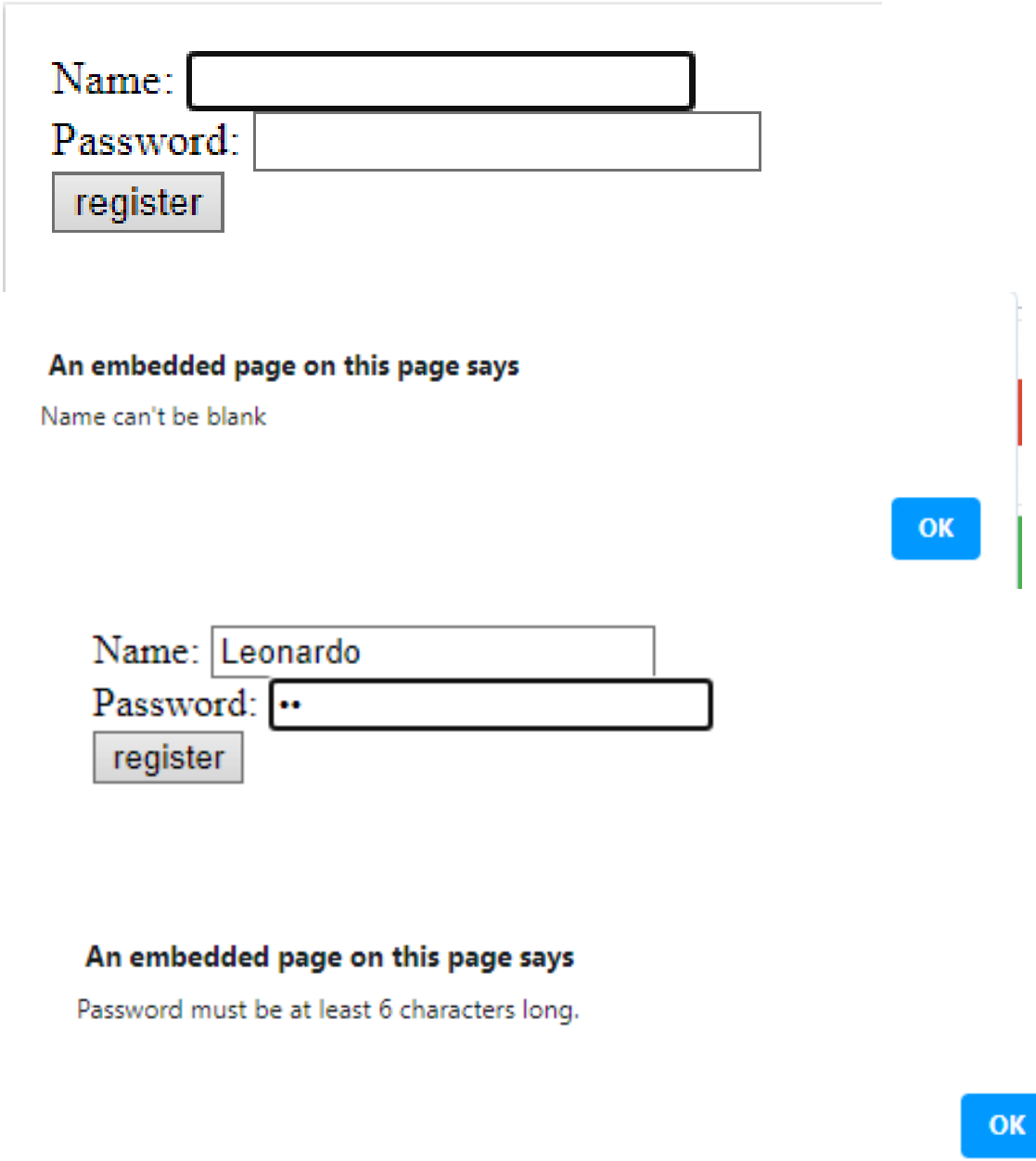
```
function validateForm() {
  var x = document.forms["myForm"]["fname"].value;
  if (x == "") {
    alert("Name must be filled out");
    return false;
  }
}
```



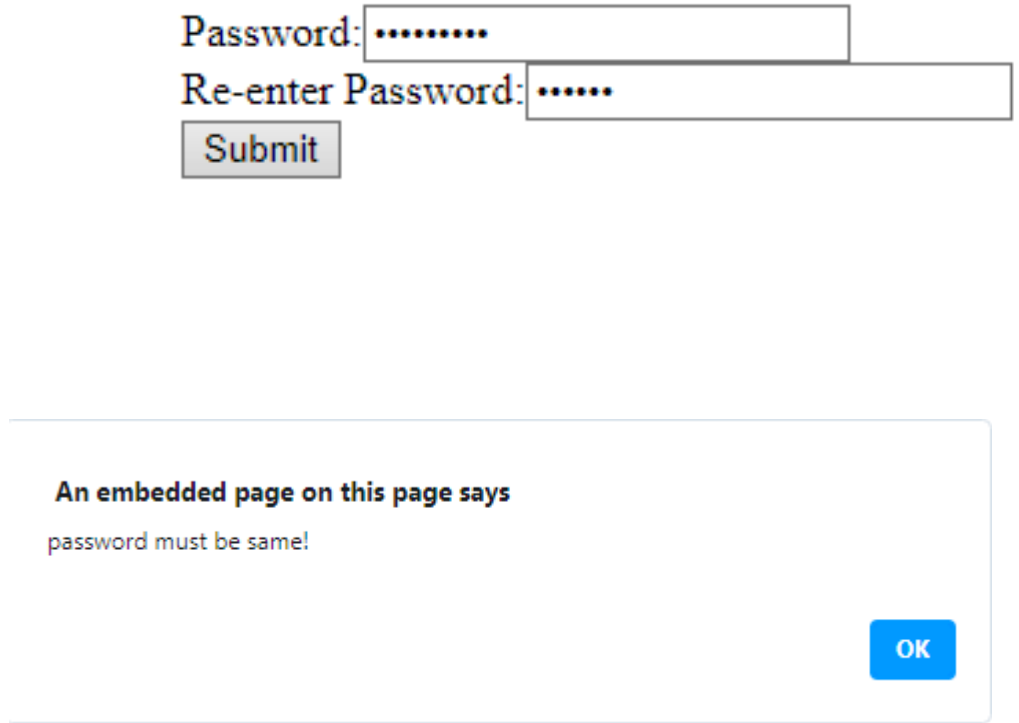
HTML Form Example

```
<form name="myForm" action="/action_page.php" onsubmit="return validateForm()" method="post">
Name: <input type="text" name="fname">
<input type="submit" value="Submit">
</form>
```

```
<html>
<body>
<script>
function validateform(){
var name=document.myform.name.value;
var password=document.myform.password.value;

if (name==null || name==""){
  alert("Name can't be blank");
  return false;
}else if(password.length<6){
  alert("Password must be at least 6 characters long.");
  return false;
  }
}
</script>
<body>
<form name="myform" method="post"
action="http://www.javatpoint.com/javascriptpages/valid.jsp"
onsubmit="return validateform()" >
Name: <input type="text" name="name"><br/>
Password: <input type="password" name="password"><br/>
<input type="submit" value="register">
</form>
</body>
</html>
```



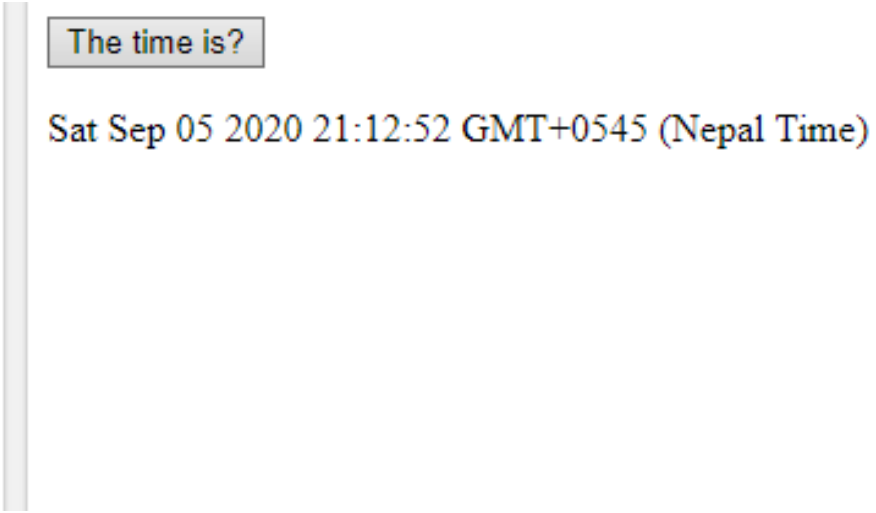
```
<html>
<head>
<script type="text/javascript">
function matchpass(){
var firstpassword=document.f1.password.value;
var secondpassword=document.f1.password2.value;

if(firstpassword==secondpassword){
return true;
}
else{
alert("password must be same!");
return false;
}
}
</script>
</head>
<body>

<form name="f1"
action="http://www.javatpoint.com/javascriptpages/valid.jsp"
onsubmit="return matchpass()">
Password:<input type="password" name="password" /><br/>
Re-enter Password:<input type="password" name="password2"/><br/>
<input type="submit">
</form>
```



JavaScript Events

- An HTML event can be something the browser does, or something a user does. Here are some examples of HTML events:
  - An HTML web page has finished loading
  - An HTML input field was changed
  - An HTML button was clicked

```
<!DOCTYPE html>
<html>
<body>

<button onclick="document.getElementById('demo').innerHTML=Date()">The time
is?</button>

<p id="demo"></p>

</body>
</html>
```



```
<!DOCTYPE html>
<html>
<body>
<h1 onclick="this.innerHTML='Ooops!'">Click on this text!</h1>
</body>
</html>
```

Click on this text! **Ooops!**

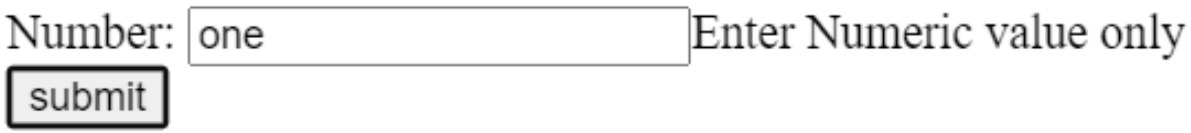
Common HTML Events

Here is a list of some common HTML events:

Event	Description
onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page

JavaScript Number Validation

```
<!DOCTYPE html>
<html>
<head>
<script>
function validate(){
var num=document.myform.num.value;
if (isNaN(num)){

document.getElementById("numloc").innerHTML=
"Enter Numeric value only";
  return false;
}else{
  return true;
  }
}
</script>
</head>
```




```
<body>
<form name="myform"
action="http://www.javatpoint.com/javascriptpages/valid.jsp"
onsubmit="return validate()" >
Number: <input type="text" name="num"><span
id="numloc"></span><br/>
<input type="submit" value="submit">
</form>

</body>
</html>
```

JavaScript email validation

```
<html>
<body>
<script>
function validateemail()
{
var x=document.myform.email.value;
var atposition=x.indexOf("@");
var dotposition=x.lastIndexOf(".");
if (atposition<1 || dotposition<atposition+2 ||
dotposition+2>=x.length){
  alert("Please enter a valid e-mail address”);
  return false;
  }
}
</script>
```

```
<body>
<form name="myform"  method="post"
action="http://www.javatpoint.com/javascriptpages/valid.jsp"
onsubmit="return validateemail();">
Email: <input type="text" name="email"><br/>

<input type="submit" value="register">
</form>
</body>
</html>
```

- email id must contain the @ and . character
- There must be at least one character before and after the @.
- There must be at least two characters after . (dot).

Lab exercises

22. Write a JavaScript Program to sort words in alphabetical order.
23. Write a Program to reverse a String
24. Write a Program to display Date and Time.
25. Write a Program to Insert Item in an Array
26. Write a Program to Remove Duplicates from an Array.
27. Write a Program to validate a form having Name, Phone Number, Email and Address.

Sort string in alphabetical order

```
// program to sort words in alphabetical order

// take input
const string = prompt('Enter a sentence: ');

// converting to an array
const words = string.split(' ');

// sort the array elements
words.sort();

// display the sorted words
console.log('The sorted words are:');

for (const element of words) {
  console.log(element);
}
```

Reverse a String

```
// program to reverse a string

function reverseString(str) {

    // empty string
    let newString = "";
    for (let i = str.length - 1; i >= 0; i--)
        newString += str[i];
    }
    return newString;
}

// take input from the user
const string = prompt('Enter a string: ');

const result = reverseString(string);
console.log(result);
```

```
function reverseString(str) {
  if (str === "")
    return "";
  else
    return reverseString(str.substr(1)) + str.charAt(0);
}
reverseString("hello");
```

```
> function reverseString(str) {
...   if (str === "")
...     return "";
...   else
...     return reverseString(str.substr(1)) + str.charAt(0);
... }
undefined
> reverseString("hello");
'olleh'
```

```
/*
First Part of the recursion method
You need to remember that you won't have just one call, you'll have several nested calls

Each call: str === "?"                              reverseString(str.subst(1))     + str.cha
1st call - reverseString("Hello")   will return     reverseString("ello")           + "h"
2nd call - reverseString("ello")    will return     reverseString("llo")            + "e"
3rd call - reverseString("llo")     will return     reverseString("lo")             + "l"
4th call - reverseString("lo")      will return     reverseString("o")              + "l"
5th call - reverseString("o")       will return     reverseString("")               + "o"

Second part of the recursion method
The method hits the if condition and the most highly nested call returns immediately

5th call will return reverseString("") + "o" = "o"
4th call will return reverseString("o") + "l" = "o" + "l"
3rd call will return reverseString("lo") + "l" = "o" + "l" + "l"
2nd call will return reverserString("llo") + "e" = "o" + "l" + "l" + "e"
1st call will return reverserString("ello") + "h" = "o" + "l" + "l" + "e" + "h"
*/
```

Remove Duplicates from an Array

```
// program to remove duplicate value from an array

function getUnique(arr){

    // removing duplicate
    let uniqueArr = [...new Set(arr)];

    console.log(uniqueArr);
}

const array = [1, 2, 3, 2, 3];

// calling the function
getUnique(array);
```

AJAX

By Yuba Raj Devkota

What is AJAX?

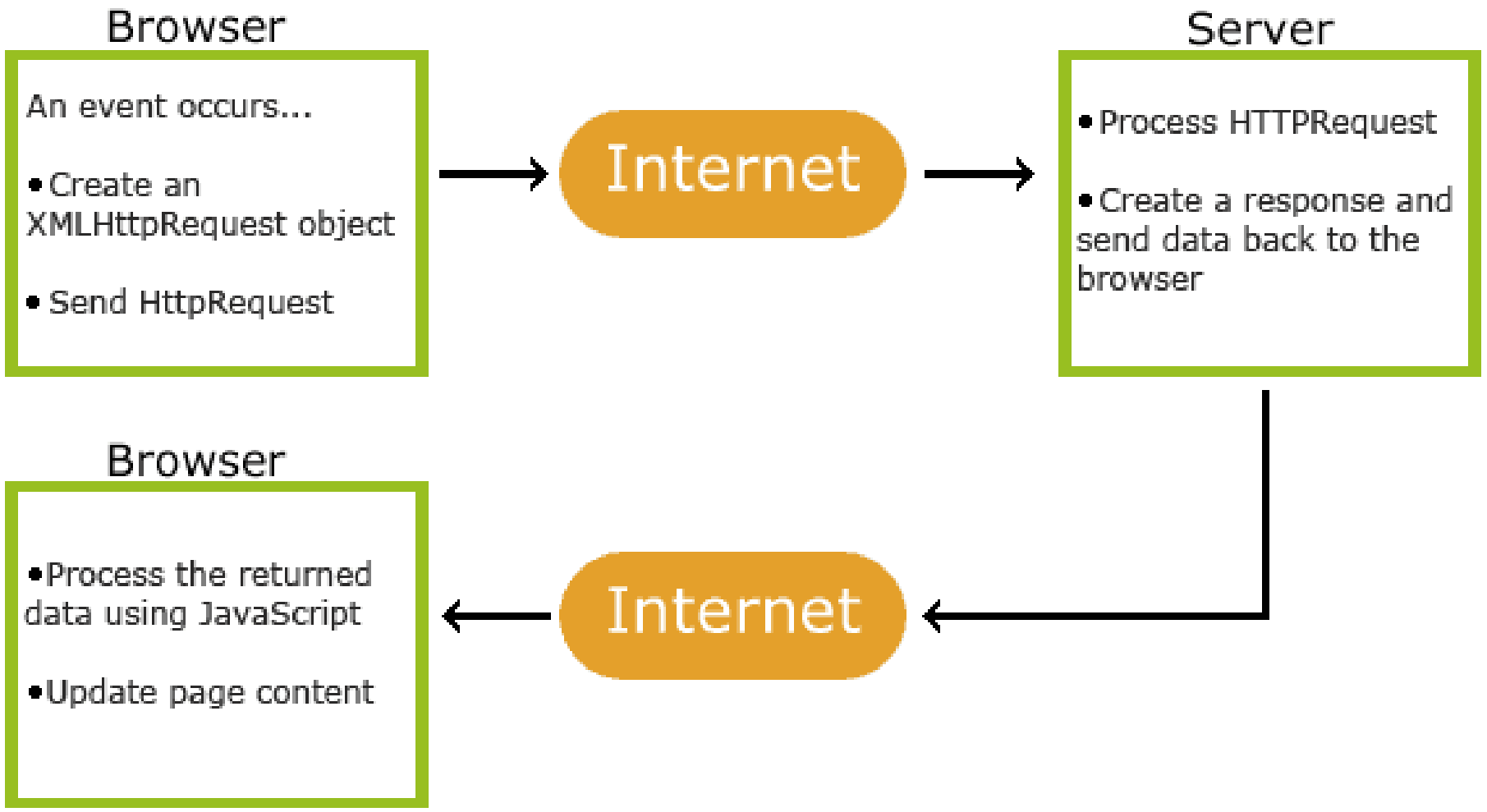
- AJAX = Asynchronous JavaScript and XML.
- AJAX is a technique for creating fast and dynamic web pages.
- AJAX is about updating parts of a web page, without reloading the whole page.
- AJAX allows web pages to be updated asynchronously by exchanging small amounts of data with the server behind the scenes. This means that it is possible to update parts of a web page, without reloading the whole page.
- Classic web pages, (which do not use AJAX) must reload the entire page if the content should change.
- Examples of applications using AJAX: Google Maps, Gmail, Youtube, and Facebook tabs.

AJAX is Based on Internet Standards

AJAX is based on internet standards, and uses a combination of:

- XMLHttpRequest object (to exchange data asynchronously with a server)
- JavaScript/DOM (to display/interact with the information)
- CSS (to style the data)
- XML (often used as the format for transferring data)

How AJAX Works



AJAX PHP Example

The following example will demonstrate how a web page can communicate with a web server while a user type characters in an input field:

Example

Start typing a name in the input field below:

First name: n

Suggestions: Nina

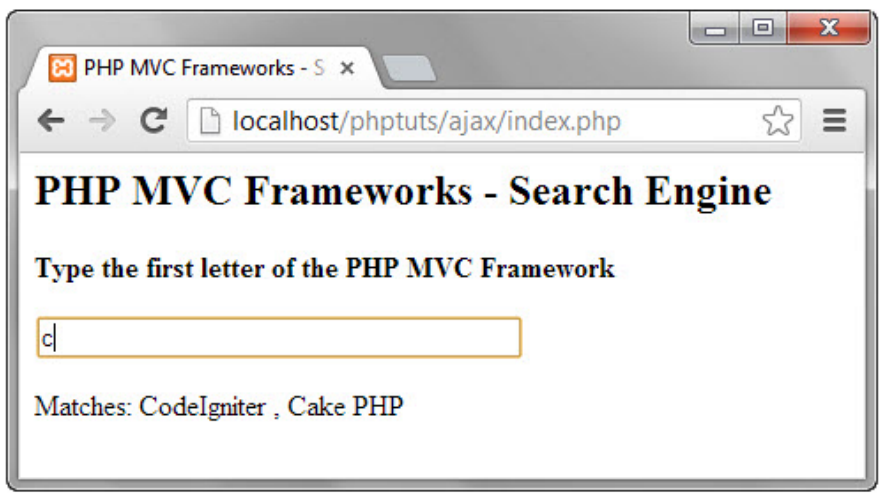
Example

Start typing a name in the input field below:

First name: np

Suggestions: no suggestion

Type the letter C in the text box you will get the following results.



The above example demonstrates the concept of AJAX and how it can help us create r

The XMLHttpRequest Object

All modern browsers support the `XMLHttpRequest` object.

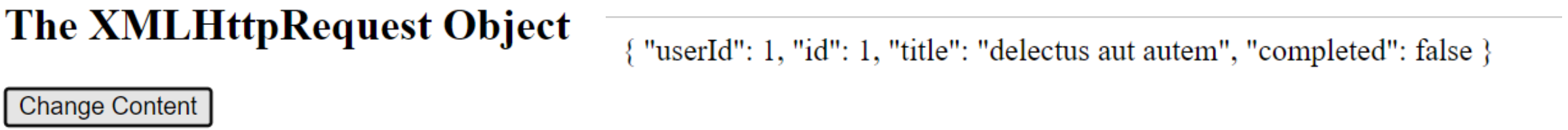
The `XMLHttpRequest` object can be used to exchange data with a web server behind the scenes. This means that it is possible to update parts of a web page, without reloading the whole page.

The keystone of AJAX is the XMLHttpRequest object.

1. Create an XMLHttpRequest object
2. Define a callback function
3. Open the XMLHttpRequest object
4. Send a Request to a server

```
<!DOCTYPE html>
<html>
<body>

<div id="demo">
<h2>The XMLHttpRequest
Object</h2>
<button type="button"
onclick="loadDoc()">Change
Content</button>
</div>
```

```
<script>
function loadDoc() {
  const xhttp = new XMLHttpRequest();
  xhttp.onload = function() {
    document.getElementById("demo").innerHTML =
    this.responseText;
  }
  var url = "https://jsonplaceholder.typicode.com/todos/1";
  xhttp.open("GET", url, true);
  xhttp.send();
}
</script>

</body>
</html>
```



JQuery

What is jQuery?

jQuery is a lightweight, "write less, do more", JavaScript library.

The purpose of jQuery is to make it much easier to use JavaScript on your website.

jQuery takes a lot of common tasks that require many lines of JavaScript code to accomplish, and wraps them into methods that you can call with a single line of code.

jQuery also simplifies a lot of the complicated things from JavaScript, like AJAX calls and DOM manipulation.

The jQuery library contains the following features:

- HTML/DOM manipulation
- CSS manipulation
- HTML event methods
- Effects and animations
- AJAX
- Utilities

Why jQuery?

There are lots of other JavaScript libraries out there, but jQuery is probably the most popular, and also the most extendable.

Many of the biggest companies on the Web use jQuery, such as:

- Google
- Microsoft
- IBM
- Netflix

Adding jQuery to Your Web Pages

There are several ways to start using jQuery on your web site. You can:

- Download the jQuery library from jQuery.com
- Include jQuery from a CDN, like Google

Downloading jQuery

The jQuery library is a single JavaScript file, and you reference it with the HTML `<script>` tag (notice that the `<script>` tag should be inside the `<head>` section):

```
<head>
<script src="jquery-3.5.1.min.js"></script>
</head>
```

jQuery CDN

Google CDN:

```
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
</head>
```

jQuery Syntax

The jQuery syntax is tailor-made for **selecting** HTML elements and performing some **action** on the element(s).

Basic syntax is: **$(*selector*).*action*()**

- A $ sign to define/access jQuery
- A (*selector*) to "query (or find)" HTML elements
- A jQuery *action*() to be performed on the element(s)

Examples:

`$(this).hide()` - hides the current element.

`$("p").hide()` - hides all <p> elements.

`$(".test").hide()` - hides all elements with class="test".

`$("#test").hide()` - hides the element with id="test".

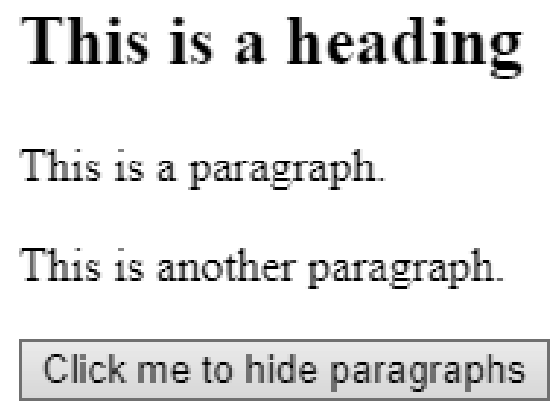
The Document Ready Event

```
$(document).ready(function(){

  // jQuery methods go here...

});
```

This is to prevent any jQuery code from running before the document is finished loading (is ready).

```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js">
</script>
<script>
$(document).ready(function(){
  $("button").click(function(){
    $("p").hide();
  });
});
</script>
</head>
<body>

<h2>This is a heading</h2>

<p>This is a paragraph.</p>
<p>This is another paragraph.</p>

<button>Click me to hide paragraphs</button>

</body>
</html>
```

Syntax	Description
$("*")	Selects all elements
$(this)	Selects the current HTML element
$("p.intro")	Selects all <p> elements with class="intro"
$("p:first")	Selects the first <p> element
$("ul li:first")	Selects the first element of the first
$("ul li:first-child")	Selects the first element of every
$("[href]")	Selects all elements with an href attribute



What is BOM in javascript? Give an example

The Browser Object Model (BOM) in JavaScript refers to the objects exposed by the web browser that allow interaction with the web browser itself, outside of the content of the web page. Unlike the Document Object Model (DOM), which deals with the document (the web page), the BOM provides objects to interact with things like the browser window, the history of the pages visited in the session, the screen dimensions, and more.

- Some of the key objects and functionalities provided by the BOM include:

1. **Window**: The global object that represents the browser window. It includes methods to control the window's appearance and behavior, such as opening new windows, resizing them, and navigating between pages.
2. **Navigator**: Provides information about the browser, such as its name, version, and the operating system it's running on.
3. **Location**: Allows the script to get or set the current URL and navigate to a new page.
4. **History**: Offers methods to interact with the browser's session history (pages visited in the tab or frame that the current page is loaded in).
5. **Screen**: Contains information about the user's screen, such as its width and height.

```
// Using the window object to open a new web page
window.open('https://www.example.com', '_blank');

// Using the location object to redirect the current page
location.href = 'https://www.example.com';

// Using the navigator object to display browser information
console.log(`Browser Name: ${navigator.appName}`);
console.log(`Browser Version: ${navigator.appVersion}`);

// Using the screen object to display screen dimensions
console.log(`Screen Width: ${screen.width}`);
console.log(`Screen Height: ${screen.height}`);

// Using the history object to go back to the previous page
history.back();
```

What is REACT? Give an example

- React is a JavaScript library developed by Facebook for building user interfaces. It allows developers to create large web applications that can change data, without reloading the page.
- The key feature of React is the concept of components: small, reusable pieces of code that represent a part of the user interface. React emphasizes a declarative approach, where you define what you want to achieve and React takes care of the DOM manipulation. It also introduces a virtual DOM to efficiently update the view when data changes.

```
import React from 'react';
import ReactDOM from 'react-dom';

function HelloMessage({ name }) {
  return <div>Hello {name}</div>;
}

ReactDOM.render(<HelloMessage name="Alice" />,
document.getElementById('app'));
```

What is Angular JS? Give an example

- AngularJS is a structural framework for dynamic web apps. It lets you use HTML as your template language and extends HTML's syntax to express your application's components clearly and succinctly. AngularJS's data binding and dependency injection eliminate much of the code you would otherwise have to write.
- It was designed to make both the development and the testing of such applications easier. However, it's worth noting that AngularJS refers to the 1.x versions of Angular, and from version 2 onwards, the framework is simply called Angular, which is a complete rewrite.

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.7.8/angular.min.js"></script>
<body>

<div ng-app="" ng-init="name='Alice'">
  <p>Hello <span ng-bind="name"></span>!</p>
</div>

</body>
</html>
```

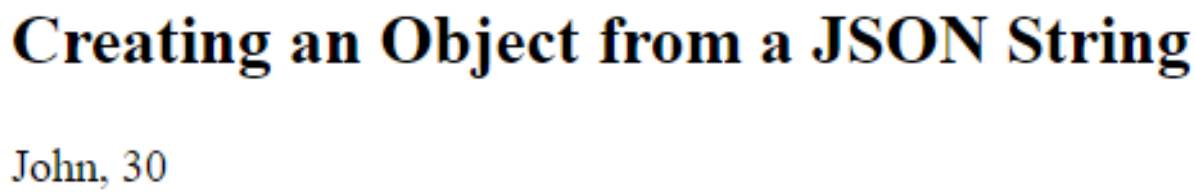
JSON

- JSON stands for **J**ava**S**cript **O**bject **N**otation
- JSON is a lightweight data-interchange format
- JSON is plain text written in JavaScript object notation
- JSON is used to send data between computers
- JSON is language independent *****

- The JSON format is syntactically similar to the code for creating JavaScript objects. Because of this, a JavaScript program can easily convert JSON data into JavaScript objects.
- Since the format is text only, JSON data can easily be sent between computers, and used by any programming language.

```
'{"name":"John", "age":30, "car":null}'
```

```
<!DOCTYPE html>
<html>
<body>

<h2>Creating an Object from a JSON String</h2>

<p id="demo"></p>

<script>
const txt = '{"name":"John", "age":30, "city":"New York"}'
const obj = JSON.parse(txt);
document.getElementById("demo").innerHTML = obj.name + ", " + obj.age;
</script>

</body>
</html>
```



```html
<!DOCTYPE html>
<html>
<body>

<h2>Create a JSON string from a JavaScript object.</h2>
<p id="demo"></p>

<script>
const obj = {name: "John", age: 30, city: "New York"};
const myJSON = JSON.stringify(obj);
document.getElementById("demo").innerHTML = myJSON;
</script>

</body>
</html>
```

Create a JSON string from a JavaScript object.

{"name":"John","age":30,"city":"New York"}

Lab exercises

28. Write an example to set cookie and display that cookie.
29. Write an example of AJAX
30. Write an example of jQuery
31. Write a program to convert JSON into JavaScript Object